

Assignment #2 Secure Authentication System

Course: Data Integrity and Authentication

Due Date: 30 April 2026 (Submission & Discussion in Section)

Team Size: 2 students per project

Task Description

You are required to develop a complete authentication system that follows real-world security practices.

The system must:

1. Allow users to register and login
2. Securely store user credentials
3. Use tokens to protect routes
4. Require 2FA verification after login
5. Implement 3 different roles with different permissions
6. Restrict access to routes based on authentication and roles

Core Requirements

1) User Registration

- User can create an account
- Required fields:
 - Name
 - Email or username
 - Password
 - Role

2) Password Hashing

- Passwords must NOT be stored as plain text
- Store only hashed passwords in the database
- During login, compare hashed values

3) Login System

- User logs in using:

- Email/username
- Password
- If correct → proceed to 2FA verification

4) Two-Factor Authentication (2FA)

- Use an authenticator app such as:
 - Google Authenticator
 - Microsoft Authenticator
 - Authy

System must:

- Generate a secret key
- Display a QR code
- Allow user to scan it
- Verify the 6-digit code entered by the user

User is authenticated ONLY if password + 2FA are correct

5) Token-Based Authentication

- After successful login + 2FA:
 - Generate a token (e.g., JWT or similar)
- Use the token to:
 - Access protected routes
 - Identify the logged-in user

System must:

- Send token to client
- Require token in protected requests
- Reject requests without valid token

6) Database

Store users in a database with fields like:

- id

- name
- email
- hashed password
- role
- 2FA secret

Any relational database is allowed.

7) Role-Based Access Control (RBAC)

You must implement exactly 3 roles:

- Admin
- Manager
- User

8) Protected Routes

- All sensitive routes must be protected using:
 - Token authentication
 - Role checking

Examples:

- Only Admin → /admin
- Only Manager → /manager
- Only User → /profile

Unauthorized access must be blocked.

9) Clean Code & Project Structure

Your project must be:

- Well-organized
- Easy to read
- Modular (separate logic into files)
- No repeated code
- Proper naming conventions

10) Git & GitHub

- Use Git during development
- Push project to GitHub
- Include multiple commits (not one upload)

System Flow (Required)

Your system must follow this flow:

1. User registers
2. Password is hashed and stored
3. 2FA secret is generated
4. QR code is displayed
5. User scans QR code
6. User logs in with password
7. User enters 2FA code
8. System verifies both
9. System generates token
10. User accesses routes using token
11. Access is controlled based on role

Minimum Required Pages / APIs

- Register
- Login
- 2FA verification
- Dashboard
- Profile page
- Admin page
- Manager page
- User page

Required Demonstration (In Section)

Each team must demonstrate:

1. Register a new user
2. Show password is hashed in DB
3. Show QR code for 2FA
4. Login process (password + 2FA)
5. Show generated token
6. Access protected routes using token
7. Show different roles:
 - Admin
 - Manager
 - User
8. Show blocked access when unauthorized
9. Show GitHub repository